

Polymorphism in C# with Real-Time Examples

In this article, I will discuss **Polymorphism in C#** with Real-Time Examples. Please read our previous discussion of [Multiple Inheritance in C#](#) with Examples. Polymorphism in C# is one of the core concepts of object-oriented programming languages (OOPs). You will understand the following pointers in detail at the end of this article.

1. [What is Polymorphism?](#)
2. [Why do we need Polymorphism?](#)
3. [Types of Polymorphism in C#?](#)
4. [What is Compile-Time Polymorphism?](#)
5. [What is Runtime Polymorphism?](#)

Note: Polymorphism is one of the primary pillars of object-oriented programming.

What is Polymorphism in C#?

Polymorphism is one of the fundamental OOP concepts and is a term used to describe situations where something takes various roles or forms. In the programming world, these things can be operators or functions.

The word polymorphism is derived from two Greek words: poly and morphs. The word “Poly” means many, and “morphs” means forms. Therefore, polymorphism means “many forms” or we can say that the word polymorphism means the ability to take more than one form. That is one thing that can take many forms.

Polymorphism is a concept by which we can perform a single task in different ways. That is, when a single entity behaves differently in different cases, it is called polymorphism in C#. The term polymorphism is an object-oriented programming term that means a function or an operator behaves differently in different scenarios.

Technically, we can say that when a function shows different behaviors when we pass different types and numbers of input values, then it is called Polymorphism in C#. So, behaving in different ways depending on the input received is known as polymorphism, i.e., whenever the input changes, automatically, the output or the behavior also changes.

We can achieve flexibility in our code using polymorphism because we can perform various operations by using methods with the same names according to our business requirements. Let's understand Polymorphism with some real-time examples.

Real-Time Examples of Polymorphism:

Let us understand a few real-time examples to understand this Polymorphism concept.

Example1:

Suppose you are in a classroom, then at that time, you will behave like a student. But when you are in the shopping mall, at that time you will behave like a customer. Again, when you are traveling on a bus, then you will behave like a passenger. And when you are at your home at that time, you will behave like a son or

daughter. Here, you are one person but performing different behaviors. This is nothing but polymorphism. The behaviors change based on the place.

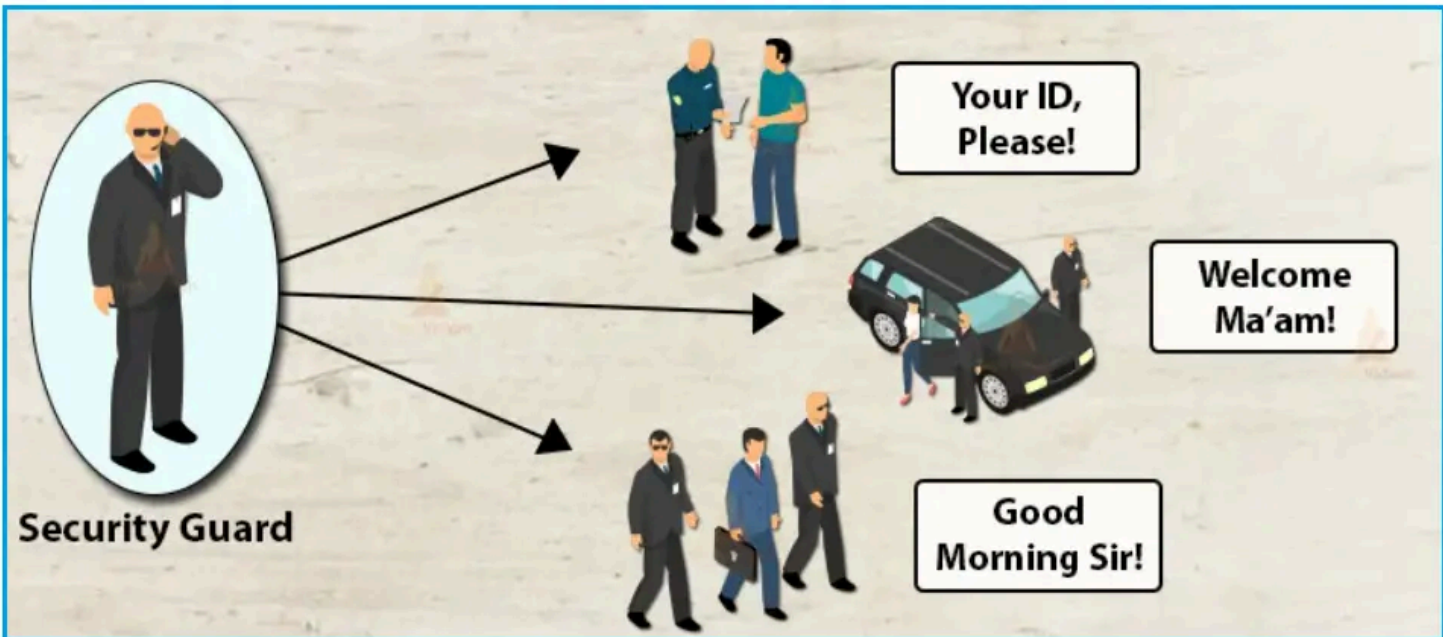


Different Behaviors:

1. In Shopping Mall, Behave Like a Customer
2. In Class Room, Behave Like a Student
3. In Bus, Behave Like a Passenger
4. In Home, Behave Like a Son or Daughter

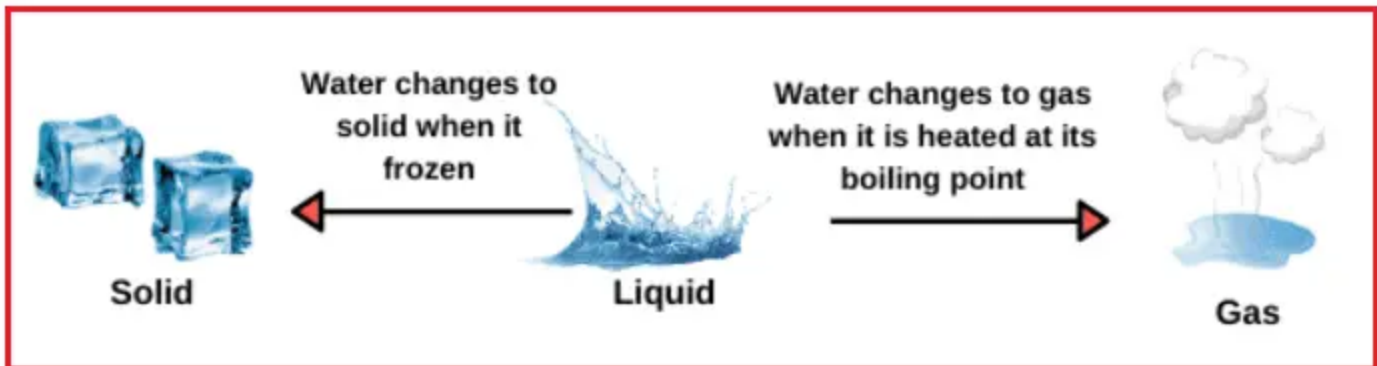
Example2:

A security guard in an organization behaves differently with different people entering the organization. The security behaves in a different way when the Boss comes and, in another way, when the employees come. And when the customers enter, the same security guard will respond differently. So here, the behavior of the security guard changes from person to person. It depends on the member who is entering the organization.



Example3:

Another good real-time example of polymorphism is water. We all know that water is a liquid at normal temperature, but it changes to a solid when it is frozen, and the same water changes to a gas when it is heated at its boiling point. This is also polymorphism.



Example4:

One of the best real-time examples of polymorphism is Women in society. The same woman performs a different role in society. The woman can be the wife of someone, the mother of her child, can be doing a job in an organization, and many more at the same time. But the Woman is only one. So, the same woman performing different roles is nothing but performing polymorphism.

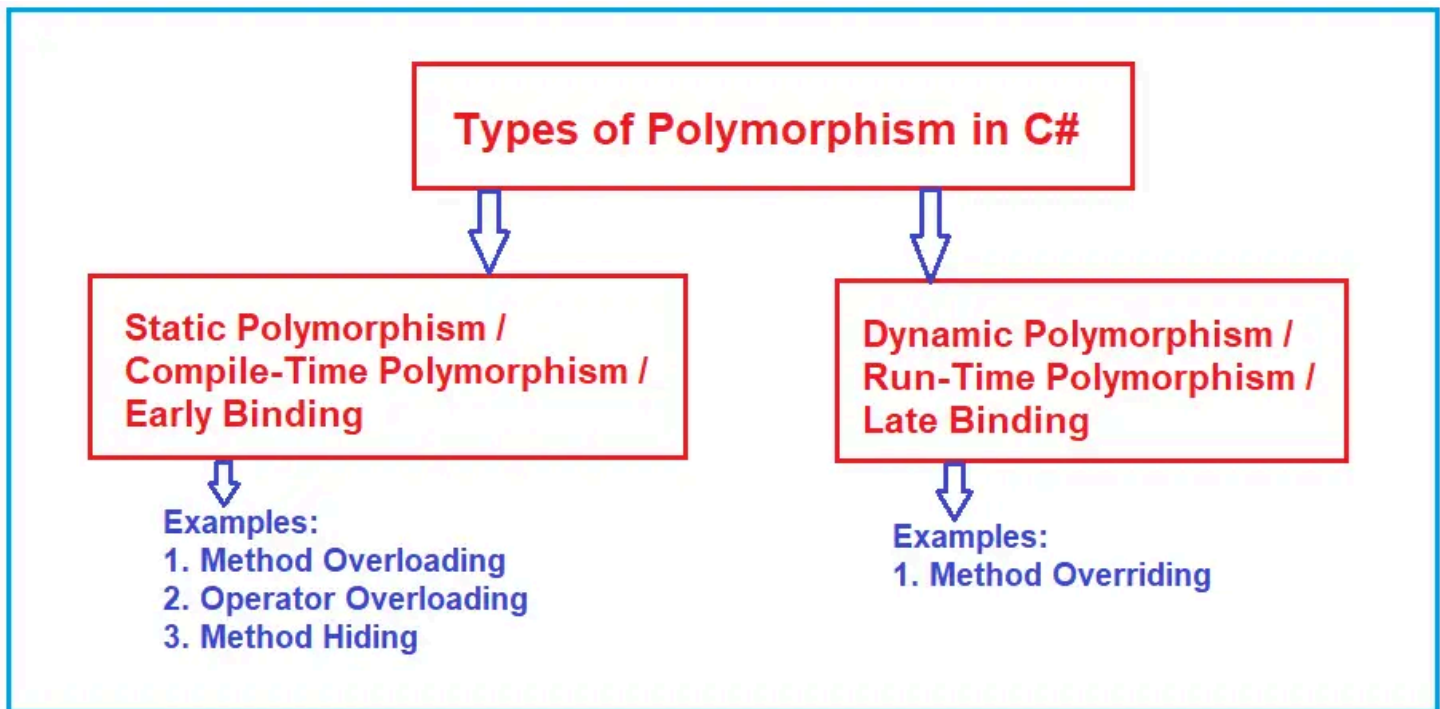


Types of Polymorphism in C#:

There are two types of polymorphism in C#. They are as follows:

1. **Static Polymorphism / Compile-Time Polymorphism / Early Binding**
2. **Dynamic Polymorphism / Run-Time Polymorphism / Late Binding**

The following diagram shows different types of polymorphisms in C# with their examples.



The polymorphism in C# can be implemented using the following three ways.

1. **Method Overloading**
2. **Operator Overloading**
3. **Method Overriding**
4. **Method Hiding**

Note: While working with Polymorphism in C#, we need to understand two things, i.e., what happens at the time of compilation and what happens at the time of execution for a given method call. Is the method going to be executed from the same class at run-time, which is bounded to the class at the compile time, or is the method going to be executed from a different class at run-time rather than the class bounded at compile time?

What is Compile-Time Polymorphism in C#?

The function call is bounded to the class at the time of compilation; if the function is going to be executed from the same bounded class at run-time, then it is called Compile-Time Polymorphism in C#. This happens in the case of Method Overloading because, in this case, each method will have a different signature, and based on the method call, we can easily recognize the method that matches the method signature.

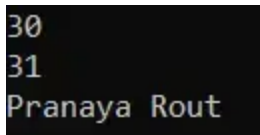
In static polymorphism, the behavior of a method is decided at compile time. That means the C# compiler binds method calls with method definition/body during compilation time only. Therefore, this type of polymorphism is also called compile-time polymorphism in C#. As the binding (the link between the function call and function definition) is performed at compile time, it is also known as early binding.

Example to Understand Compile Time Polymorphism in C#:

In the below example, within the Program class, we have defined three overloaded versions of the Add method but with different signatures. In this case, at compilation time only, we will come to know, and the method binding is resolved at runtime, which method is going to be executed. If this is not clear at the moment, then don't worry; we will discuss this concept in detail in our [Method Overloading](#) article.

```
using System;
namespace MethodOverLoading
{
    class Program
    {
        public void Add(int a, int b)
        {
            Console.WriteLine(a + b);
        }
        public void Add(float x, float y)
        {
            Console.WriteLine(x + y);
        }
        public void Add(string s1, string s2)
        {
            Console.WriteLine(s1 + " " + s2);
        }
        static void Main(string[] args)
        {
            Program obj = new Program();
            obj.Add(10, 20);
            obj.Add(10.5f, 20.5f);
            obj.Add("Pranaya", "Rout");
            Console.ReadKey();
        }
    }
}
```

Output:



```
30
31
Pranaya Rout
```

What is Runtime Polymorphism in C#?

In dynamic polymorphism, the behavior of a method is decided at runtime. Therefore, the CLR (Common Language Runtime) binds the method call with the method definition/body at runtime and invokes the relevant method during runtime when the method is called.

The function call is bounded to the class at the time of compilation; if the function is going to be executed from a different class at run-time rather than the class bounded at compilation time, then it is called Run-Time Polymorphism. This happens in the case of Method Overriding because, in this case, we have multiple methods with the same signature, i.e., the Parent Class and the Child class have the same method

implementation. So, in this case, we will be able to know at runtime from which class the method is going to be executed.

It is also called Dynamic Polymorphism or Late Binding, as at Run-time, we will be able to know from which class the method is going to be executed.

Example to Understand Dynamic Polymorphism in C#:

In the below example, we have created one virtual method in class Class1, and we have re-implemented that method inside class Class2. That means the same Show method implementation is available in both Parent and Child classes. In the Main method, we are creating an instance of the child class but storing that instance in the Parent class reference variable; in this case, from which class the Show method to be executed will be decided at runtime only. This is nothing but dynamic polymorphism. If this is not clear at the moment, then don't worry; we will discuss this concept in detail in our [Method Overriding](#) article.

```
using System;
namespace PolymorphismDemo
{
    class Class1
    {
        //Virtual Function (Overridable Method)
        public virtual void Show()
        {
            //Parent Class Logic Same for All Child Classes
            Console.WriteLine("Parent Class Show Method");
        }
    }
    class Class2 : Class1
    {
        //Overriding Method
        public override void Show()
        {
            //Child Class Reimplementing the Logic
            Console.WriteLine("Child Class Show Method");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Class1 obj1 = new Class2();
            obj1.Show(); //Resolve at Runtime

            Console.ReadKey();
        }
    }
}
```

Output: **Child Class Show Method**

In the next article, I will discuss [Method Overloading in C#](#) with Real-time Examples. Here, in this article, I try to explain **Polymorphism in C#** and its types and when to use Polymorphism in C# with Real-time Examples.